



FACULTY OF COMPUTING

SEMESTER 2 2025/2026

SECP3843 - SPECIAL TOPIC IN DATA ENGINEERING

SECTION 2

LECTURER: DR. ARYATI BINTI BAKRI

TUTORIAL 4 - GAI [AI-Assisted ETL Data Pipeline Using an AI Agent]

GROUP 17

STUDENT NAME	MATRIC NO
CHOH JING YI	A23CS0296
TAN ZHI MING	A23CS0189
LAU YAN KAI	A23CS0098

WeatherAPI to Firebase Pipeline

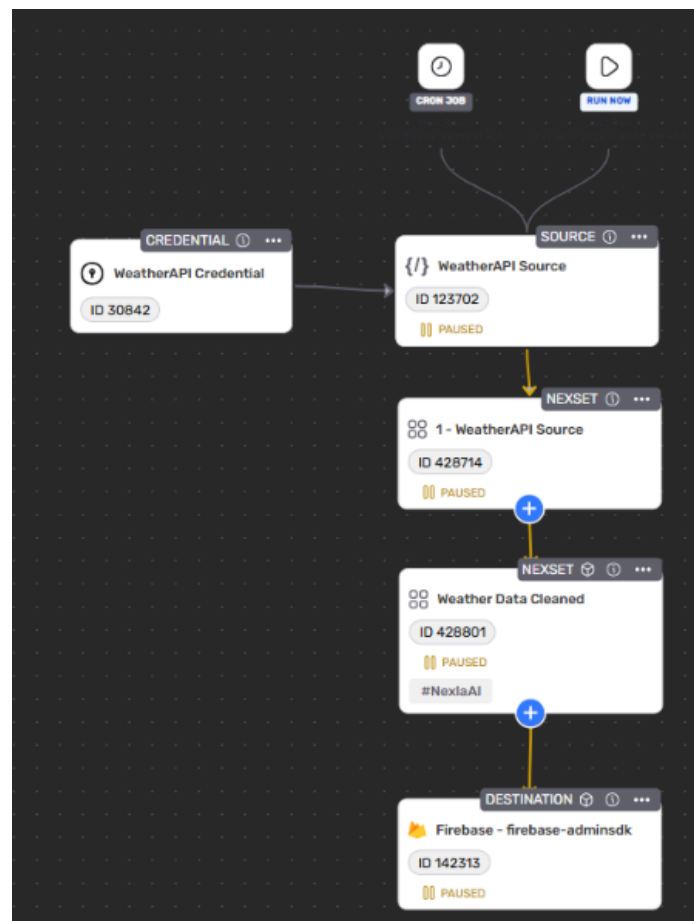
1. Introduction

This report is based on Tutorial 5, “ETL data pipeline with an AI Agent”. This tutorial demonstrates how AI agents can use natural language prompts to help create a complete ETL data pipeline. In this project, the tutorial concept is applied by building a weather data pipeline, which extracts current weather data from WeatherAPI.com, converts the response into a strict Firestore-compatible JSON format, and loads the results into Firebase Firestore.

The final pipeline follows this flow:

WeatherAPI.com → Extract Data → Transform Weather JSON → Load to Firebase Firestore

The pipeline can be run manually using “Run Now” or automatically through planned cron jobs.



2. What is AI-Assisted Data Engineering?

AI-assisted data engineering is the use of artificial intelligence tools or artificial intelligence agents to support data engineering tasks, such as pipeline design, data ingestion, conversion, debugging and workflow automation. Users can use prompts to describe the required pipeline instead of manually configuring each component or writing all the code from the beginning.

In modern data engineering, artificial intelligence assistance is useful because data pipelines often involve repetitive tasks such as connecting to APIs, selecting fields, cleaning up data, mapping patterns, and loading data into storage. AI agents can reduce development time by suggesting appropriate pipeline steps and generating conversion logic. However, AI-assisted data engineering has not eliminated the need for human understanding. Engineers still need to verify API credentials, check the output data, identify false assumptions, and confirm whether the pipeline produces the correct results.

3. Role of AI Agent in the Tutorial

In this project, AI agent acted as a pipeline construction assistant. AI agent help convert natural language requests into valid ETL pipelines.

The AI agent was used to:

1. Create a connection to WeatherAPI.com
2. Validate the API source
3. Discover available fields from the API response
4. Create a transformation step to clean and flatten the weather data
5. Suggest Firestore as the destination database.
6. Connect the source, transform, and sink into one pipeline.
7. Support manual and scheduled pipeline execution.

We selected Firebase Firestore as the destination because it is free, document-based, and highly scalable.

4. Pipeline Implementation

4.1 Extract

The pipeline extracts the current weather data from [WeatherAPI.com](https://weatherapi.com). The source endpoint retrieves the real-time weather information of Johor Bahu in Malaysia. The extracted raw data includes location and weather information, such as city, region, country, temperature, humidity, wind speed, conditions, pressure, precipitation, cloud cover and ultraviolet index.

4.2 Transform

The original WeatherAPI response adopts nested JSON format. This structure is not suitable for direct analysis, because some fields are stored in nested objects, such as position, current and conditions. The AI agent has created a transformation called “weather data flattening”. This transformation extracts useful fields and converts nested JSON into a flat JSON structure. The final conversion field is:

Field	Description
city	City name
region	State or region
country	Country name
local_time	Local weather location time
last_updated	WeatherAPI update time
temp_c	Temperature in Celsius
temp_f	Temperature in Fahrenheit
condition_text	Weather condition
humidity	Humidity percentage
wind_kph	Wind speed
pressure_mb	Air pressure
precip_mm	Precipitation

Field	Description
cloud	Cloud percentage
feelslike_c	Feels-like temperature
uv	UV index
ingested_at	Pipeline ingestion timestamp

4.3 Load

The transformed data is loaded into Firebase Firestore.

The output location is:

Item	Value
Project	auto-weather-fetch
Collection	weatherReport
Storage Type	Document-based database

Each pipeline run inserts one weather document into the weather_data collection.

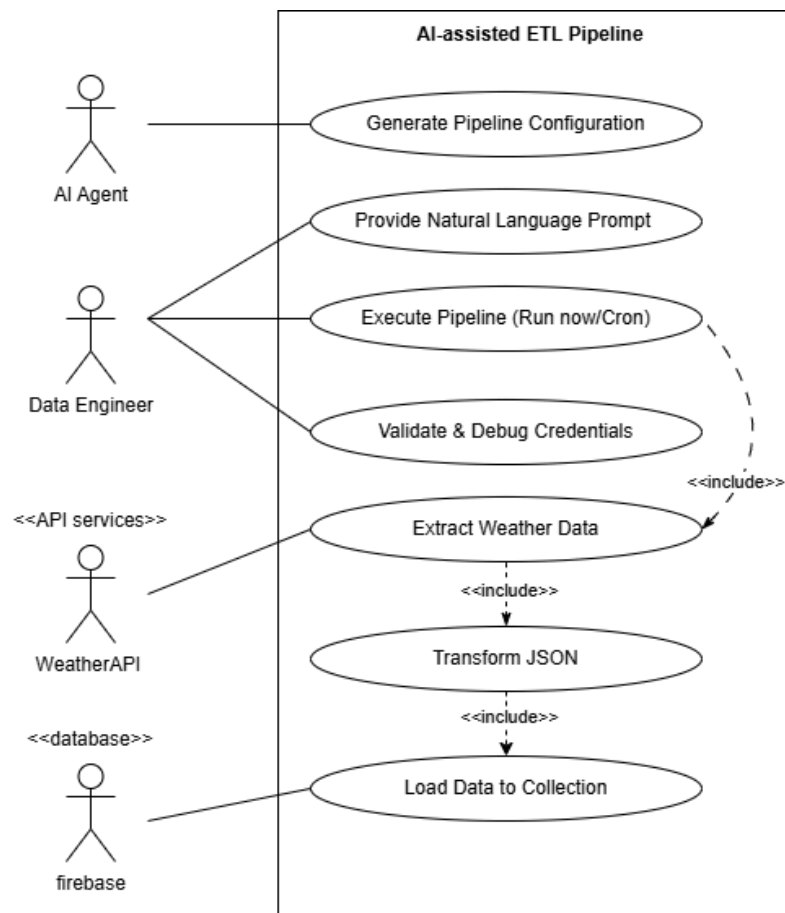
4.4 Schedule/Cron

The pipeline supports two execution methods:

Method	Purpose
Run Now	Manually execute the pipeline for testing
Cron Job	Automatically run the pipeline based on schedule

The scheduled execution allows the pipeline to collect updated weather records repeatedly based on the user settings. For example, we are fetching the weather data and insert into the collection every 2 minutes.

5. Use Case Diagram



Actor	1. <u>Data Engineer (Human)</u> The main users responsible for initiating the pipeline creation, managing security credentials, and controlling the execution plan. 2. <u>AI Agent (System)</u> An automatic AI assistant that can convert engineers' requests into actual technical configurations. 3. <u>WeatherAPI (External System)</u> Data source provider 4. <u>Firebase (External System)</u> Destination storage
--------------	--

Use Cases	<u>Pipeline Setup Phase</u> • Provide Natural Language Prompt Data engineers input their pipeline requirements in plain text. • Generate Pipeline Configuration AI agent automatically builds the architecture of the pipeline based on the instructions. • Validate & Debug Credentials Data engineer checks manually to ensure that the connection keys of the API and the database were correct. <u>Pipeline Execution Phase</u> • Execute Pipeline (Run now/Cron) Data engineers trigger the start of the process either by manually running it or by setting up automatic cron tasks at regular intervals. • Extract Weather Data This pipeline is connected to the external WeatherAPI to obtain the raw data. • Transform JSON The system will clean up and reformat the JSON data. • Load Data to Collection The converted data is then pushed to the external Firebase database for storage.
------------------	--

6. Prompt and Code / Logic Explanation

The main prompt used was:

“I want to create an ETL data pipeline from WeatherAPI.com to a document-based database. The pipeline should fetch current weather data for Johor Bahru, transform the response into a clean JSON format, and store each record in a database collection. The pipeline should run every 2 minutes.”

A second prompt was used for transformation:

“Please clean the WeatherAPI response and extract only useful fields into a simple JSON structure: city, region, country, local_time, last_updated, temp_c, temp_f, condition_text, humidity, wind_kph, pressure_mb, precip_mm, cloud, feelslike_c, uv, and ingested_at.”

The transformation logic can be represented as:

```
weather_record = {
    "city": response["location"]["name"],
    "region": response["location"]["region"],
    "country": response["location"]["country"],
    "local_time": response["location"]["localtime"],
    "last_updated": response["current"]["last_updated"],
    "temp_c": response["current"]["temp_c"],
    "temp_f": response["current"]["temp_f"],
    "condition_text": response["current"]["condition"]["text"],
    "humidity": response["current"]["humidity"],
    "wind_kph": response["current"]["wind_kph"],
    "pressure_mb": response["current"]["pressure_mb"],
    "precip_mm": response["current"]["precip_mm"],
    "cloud": response["current"]["cloud"],
    "feelslike_c": response["current"]["feelslike_c"],
    "uv": response["current"]["uv"],
    "ingested_at": current_utc_timestamp
}
```

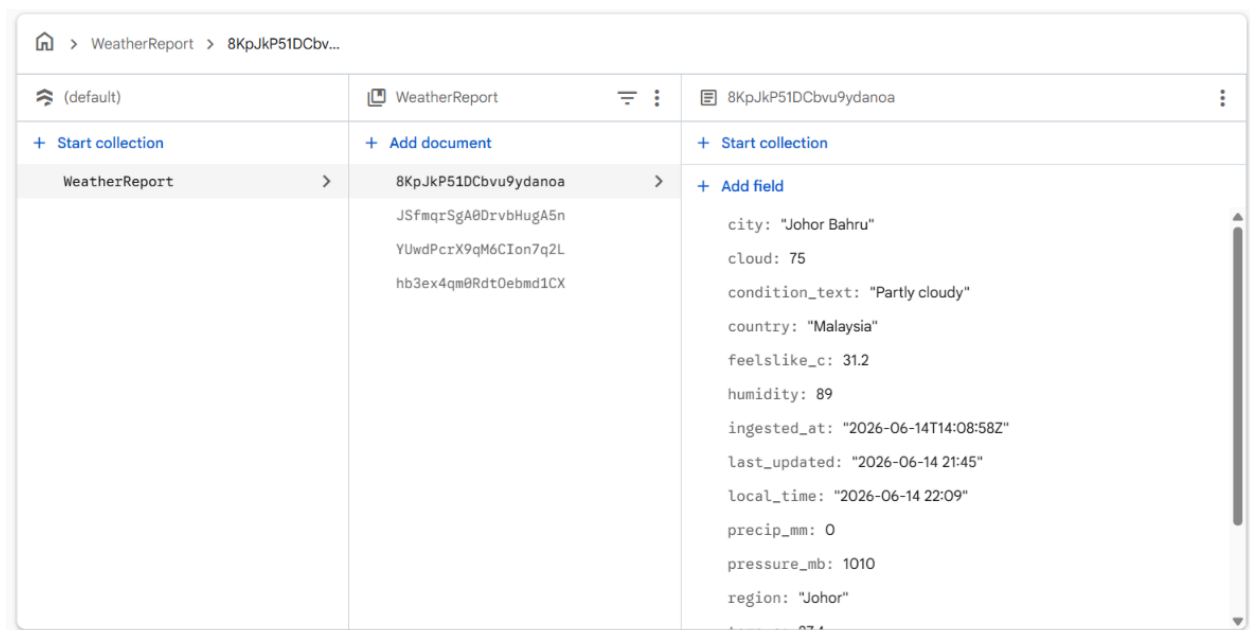
This logic shows how the pipeline transforms nested API data into a clean output document.

7. Results and Explanation

The pipeline successfully loaded transformed weather records into Firebase.

The final output was stored in the WeatherReport collection.

A sample output document is:



```
{
  "country": "Malaysia",
  "city": "Johor Bahru",
  "region": "Johor",
  "local_time": "2026-06-13 13:38",
  "last_updated": "2026-06-13 13:15",
  "temp_c": 32.2,
  "temp_f": 90,
  "condition_text": "Partly cloudy",
  "humidity": 71,
  "wind_kph": 6.8,
  "precip_mm": 0.55,
  "cloud": 75,
  "feelslike_c": 39.2,
  "uv": 9.3,
  "ingested_at": "2026-06-13T05:38:31Z"
}
```

This result proves that the ETL process worked correctly:

ETL Stage	Result
Extract	Weather data retrieved from WeatherAPI
Transform	Nested JSON converted into clean flat JSON

Load	Transformed records stored in Firebase
Schedule	Multiple records created through repeated runs

8. Reflection

Tan Zhi Ming - A23CS0189

It's good to gain more understanding of how AI agents can be used to assist with data engineering tasks. The AI agent was used to assist in the pipeline development process, including helping to set up the source, transformation, and destination. It also helped in identifying the right fields in the WeatherAPI response and setting up an elegant output format. An advantage of using an AI agent is quicker development. I didn't have to write all the code for extracting and transforming the API; I could just say it in prompts. The AI agent then created the pipeline flow and recommended appropriate configurations.

This project also revealed that there was a need for human validation in the world of AI. It was noted that the WeatherAPI key was shown as an invalid API key when added during setup, but it was successful when run in the browser. The key wasn't the problem; it was how the API credentials were set up. This is a good example of how engineers must learn to handle debugging, API parameters, and authentication. The other restriction is tool dependency. It was originally designed to be used with Firebase, but the other document-based sink was only available for MongoDB. This was a real eye-opener for me and something I needed to be aware of when using the tools and making sure that the objective of the pipeline was not lost. Future improvements are to add more cities' weather data, add data quality checks, add a dashboard, and add alerts for high temperature, heavy rainfall, or high UV index.

Lau Yan Kai - A23CS0098

When WeatherAPI is integrated with Firebase Firestore, I realised that just extracting the original JSON data is usually not enough. The strict schema of the target database must determine the conversion stage. Therefore, I had to design a custom Python layer, format the data as a Firestore document mask, and generate a flat composite document ID. This experience

taught me how to optimise the collection structure to support front-end query and avoid performance problems caused by deep nesting.

On the positive side, Claude as an efficient diagnostic partner, helped me explain the “400 error requests” error in time and adjusted the plan when the initial data loading failed. However, its main limitation is that it initially generated standard JSON packets based on general NoSQL assumptions without taking into account the specific REST API of Firestore. This shows that although AI can accelerate the coding process, human engineers still need to fully master the field knowledge of the target system to verify the output and correct the blind spots in the AI architecture.

Next, I have a practical way to improve this process and my AI workflow. From a technical point of view, I plan to expand the extraction stage, process multiple cities at the same time, and implement automatic data quality checks such as processing empty values before data loading. Regarding the use of AI, I will enhance my prompt engineering ability by clearly specifying the strict pattern constraints of the target database in the first prompt. For example, “format the output as a specific Firestore document mask package”. By setting these architectural boundaries in advance, it will greatly reduce the debugging cycle and make future AI collaboration more efficient.

Choh Jing Yi- A23CS0296

In this tutorial, it was a great experience to learn creating an ETL pipeline using Claude AI. The AI agent was very excellent at helping to handle boilerplate tasks such as formulating the API extraction logic and flattening the nested JSON response. However, AI can quickly produce working code but it lacks the understanding of context needed for the development of a robust system. The problems we had with the hierarchical structure of Firestore made us realize that no matter what, humans will need to continue playing an important role in deciding the structure of documents, modeling data and debugging the credential configurations.

I plan to apply these AI-assisted workflows to my own technical interest, such as streamlining the Python applications I use to track global market trends. Using AI agents to handle repetitive data ingestion from financial APIs allows me to focus on more intricate analysis, extensive cloud setup, and dashboard optimization. This tutorial showed me that AI is

not the end goal of data engineering, but rather a means to make me think more like a system architect rather than just a data engineer who knows how to write some basic syntax.

For future improvements, we recommend implementing standardized instruction guidelines to provide clear architectural rules before any code generation takes place. In addition, AI-generated pipelines should automatically include comprehensive data quality validations, such as null checks, duplicate detection, and data consistency verification. These enhancements would help ensure that data workflows remain reliable, efficient, and suitable for production environments while maximizing the benefits of AI-assisted development.

9. Conclusion

The project successfully implemented the AI-assisted ETL pipeline using WeatherAPI and Firebase Firestore. AI agents help establish pipelines from source configuration to conversion and destination settings. The final output in Firebase confirms that the pipeline correctly extracts, converts and loads the weather data. AI-assisted data engineering can improve productivity and reduce the workload of manual settings. However, manual verification is still important to ensure that the pipeline is correct, reliable and suitable for the project goals.